CodeOpinion

When NOT to write an Abstraction Layer

by Derek Comartin / January 12, 2022

Sponsor: *Do you build complex software systems? See how NServiceBus makes it easier to design, build, and manage software systems that use message queues to achieve loose coupling. Get started for free.*

Learn more about Software Architecture & Design.
Join thousands of developers getting weekly updates to increase your understanding of software architecture and design concepts.

Email Address Subscribe

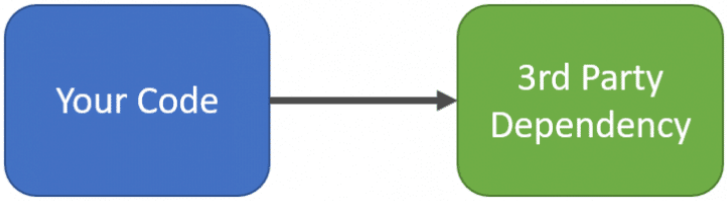
Common advice is to abstract dependencies. Often this is because people have been burned by depending directly upon 3rd party dependencies that they are highly coupled to. If they need to change the dependency, which they are highly coupled with, it can be pretty painful. This is why people say to create an abstraction layer around any dependency to isolate it so that your application code is coupled to yourself rather than a 3rd party. While this is generally true, it depends, and you don't want to create an abstraction that is more work in the short and long run by creating the wrong abstraction. Here's a concrete example of when you shouldn't abstract.

YouTube

Check out my YouTube channel where I post all kinds of content that accompanies my posts including this video showing everything that is in this post.

Abstraction Layer

So why is the general advice to abstract 3rd party dependencies? Well if you're going to be highly coupled to the dependency, and it has breaking changes (via new releases/versions), then you're going to have to deal with all the breaking changes everywhere you're using that dependency in your application code.

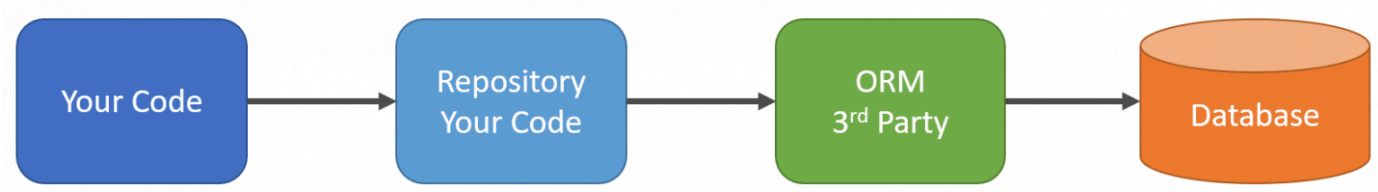


To alleviate that issue, one solution is to create your own abstraction layer that your application code depends on instead. This way if there is any change to the underlying dependency, you are changing it within the implementation of your abstraction.



Generally, this abstraction is a wrapper (facade) that hides the complexity of the 3rd party and sometimes simplifies its usage for your specific use case(s).

A common example to think of is the Repository Pattern. You're creating a repository to abstract the native interactions with the underlying database. These interactions could be from a 3rd Party ORM or even using the native SDK for the database you're using.



In this case, your application code is not directly coupled to the ORM (dependency) but rather your application code is coupled to your repository. If anything changes with the underlying ORM (or SDK) then you must make changes to the repository rather than in any application code.

I've talked about the repository pattern before, you might want to check out my post Should you use the Repository Pattern? With CQRS, Yes and No!

Messaging

While creating abstractions around 3rd parties is generally a good idea in a lot of situations, it's not a very good idea in others. A concrete example of this is with a messaging library.

If you're using a specific message broker directly with their SDK, then you likely will create for the same reason create an abstraction around it.



However, once you get into the deep end with messaging and are using an Event (Message) Driven Architecture, you'll start implementing common patterns and concepts.

Common patterns and concepts such as the Outbox Pattern, Fault Tolerance, Scheduled (Delayed) Delivery, Claim Check, Encryption, Stateful(less) Process Managers, and more.

However, there are messaging libraries that already do this. They are themselves abstractions over various queuing technology. Meaning you can use a messaging library that would support RabbitMQ, Azure Service Bus, Amazon SQS, and others.

Messaging libraries also implement and expose ways to use those patterns and concepts that are very common when in an Event (Message) Driven Architecture.

So the question becomes, should you abstract a messaging library?



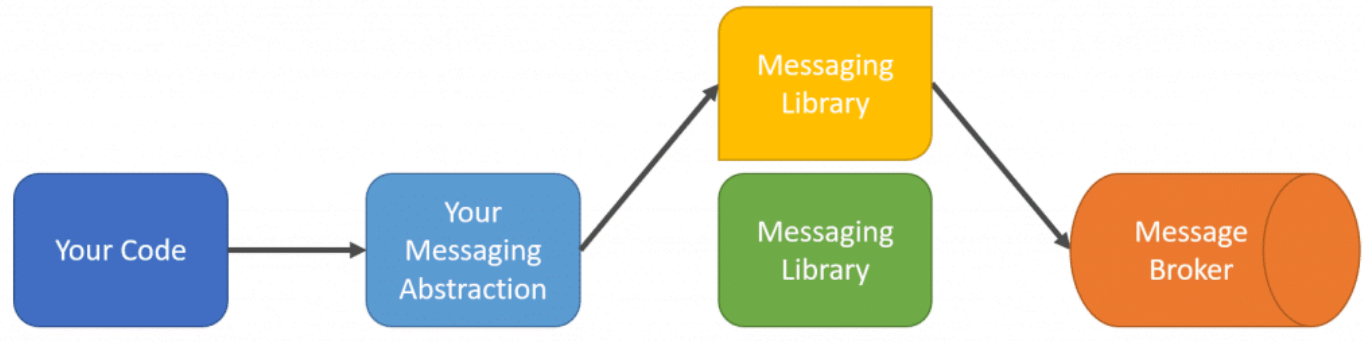
Opinionated

The problem is these types of libraries are very opinionated on how they implement common patterns and concepts. This means that if you create your own abstraction over a messaging library, you'll inherently create something that is very specific to the actual messaging library.

If you need to expose something like the outbox pattern in your own abstraction, then depending on how the underlying messaging library accomplishes that, you'll have to expose that in a very similar way.

Meaning you'll likely leak some details that are not implemented the same way within another messaging library.

Your abstraction will have the same similar API surface and opinions about implementation as the underlying messaging library. This means your abstraction will be very similar to the point of adding no additional value other than hiding the dependency.



Now if you want to change the messaging library dependency with another, you might think that having your abstraction would be useful. But since it's very opinionated, they don't really line up in terms of how functionality is exposed. I purposely made the two Messaging Library boxes in ıgrams different shapes for this reason.

You won't be able to fit your abstraction over a different dependency because both dependencies are different in how they accomplish the same patterns and concepts.

I'm not against abstractions or creating your own abstraction layers over dependencies. I do think in general rule is a decent approach in a lot of situations, just not all situations. Creating your own abstraction so you depend on your own types, often facades that isolate and simplify the underlying dependency for your use cases.

Complexity

In the concrete example of a messaging library, it doesn't make sense to create an abstraction layer. A messaging library is itself an abstraction. It will be core to a system that is built around Event Driven Architecture and messaging. There will be little value gained by creating your own abstraction but rather more complexity added by adding indirection.

Source Code

Developer-level members of my YouTube channel or Patreon get access to the full source for any working demo application that I post on my blog or YouTube. Check out the YouTube Membership or Patreon for more info.

Related Links

- What's the Cost of Indirection & Abstractions?
- Should you use the Repository Pattern? With CQRS, Yes and No!
- My TOP Patterns for Event Driven Architecture

Follow @CodeOpinion

Learn more about Software Architecture & Design.
Join thousands of developers getting weekly updates to increase your understanding of software architecture and design concepts.

Email Address Subscribe

PREVIOUS
CQRS & Event Sourcing Code Walk-Through

NEXT
Securing Sensitive Data in an Event Driven Architecture

Neve | Powered by WordPress